Appendix A: Mathematical Application of Gradient Descent in Taxon Invoice Matching

Taxon compares a purchase register with a GSTR-2B statement. This appendix explains the mathematics of the one component trained by **gradient descent**: a logistic-regression model that estimates whether two invoice records are the same transaction. Everything below describes code that exists in the project (section A.9 names the files). The model is trained on **synthetic, fictional** invoice pairs only.

A.1 Objective of the model

Each candidate pair has one record from the purchase register and one from GSTR-2B. The target is a label $y \in \{0, 1\}$: $y = 1$ if both records describe the same invoice, $y = 0$ otherwise. Logistic regression turns the pair's features into a number $\hat{y} \in (0, 1)$, the *model-predicted match probability*. It is uncalibrated, so it is a score to rank and flag pairs, not a guarantee. Taxon uses it beside the rule-based engine; it never silently overrides a status.

A.2 Feature representation

Each pair becomes a vector $\mathbf{x} = [x_1, x_2, \ldots, x_{13}]^\mathsf{T}$:

Feature	Definition	Range
`supplier`	Supplier-name similarity: the better of Jaro-Winkler on the cleaned names and on their alphabetically sorted words	[0,1]
`invoice`	1 if the normalised invoice numbers are identical, else 1 – d/max length (Levenshtein distance d)	[0,1]
`date`	max(0, 1 – days apart / 30)	[0,1]
`amount`	1 if the totals are within ₹1, else 1 – |difference| / larger total	[0,1]
`gstin`	GSTIN indicator: 1 if equal, 0 if both present and different	{0,1}
`supplier_missing` … `gstin_missing`	Five indicators: 1 when that field is missing on either record (the matching similarity is then set to 0)	{0,1}
`amount_rel_diff`	min(|difference| / larger total, 1)	[0,1]
`date_diff_scaled`	min(days apart, 30) / 30	[0,1]
`invoice_exact`	1 if the normalised invoice numbers are identical	{0,1}

Missing values. A similarity that cannot be computed (a field is absent) is set to 0 and its `*_missing` indicator is set to 1, so the model can learn that "missing" is different from "dissimilar". **Scaling.** Using only the training set, each feature is standardised: $\tilde{x}_j =$

$\frac{x_j - \mu_j}{\sigma_j + 10^{-9}}$, where μ_j and σ_j are the training mean and population standard deviation. The same μ, σ are stored with the model and applied to new pairs. From here on $\tilde{x}$ is simply written x.

A.3 Prediction function

The linear combination of features is

$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_{13} x_{13} + b,$$

where $\mathbf{w}$ holds the trainable weights and b is the bias. Because z can be any real number, it is passed through the sigmoid:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

For every real z, $e^{-z} > 0$, so the denominator exceeds 1 and $0 < \hat{y} < 1$. Also $\sigma(0) = 0.5$, σ increases with z, and $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$. The implementation evaluates $e^{-|z|}$ so the exponential never overflows.

A.4 Loss function

For one example with true label y and prediction $\hat{y}$, the binary cross-entropy is

$$L = -\big[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})\big].$$

If $y = 1$ this is $-\ln \hat{y}$ (small when $\hat{y}$ is near 1, large when near 0); if $y = 0$ it is $-\ln(1 - \hat{y})$. Over m training pairs the loss to minimise is

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \Big[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \Big].$$

Here m is the number of pairs, y_i the label of pair i, $\hat{y}_i = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b)$ its prediction, $\ln$ the natural logarithm. J is the negative average log-likelihood of the labels, so minimising J is the same as maximum-likelihood estimation. Predictions are clipped to $[10^{-12}, 1 - 10^{-12}]$ so that $\ln 0$ never occurs.

A.5 Derivation of the gradients

Take one example. We need $\partial L/\partial w_j$ and $\partial L/\partial b$. The chain of dependence is $w_j, b \to z \to \hat{y} \to L$, so the chain rule gives $\dfrac{\partial L}{\partial w_j} = \dfrac{\partial L}{\partial \hat{y}} \cdot \dfrac{\partial \hat{y}}{\partial z} \cdot \dfrac{\partial z}{\partial w_j}$.

Step 1. Differentiate $L = -[y \ln \hat{y} + (1-y)\ln(1-\hat{y})]$ with respect to $\hat{y}$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}.$$

Step 2. Since $\hat{y} = \sigma(z)$,

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}).$$

Step 3. Multiply:

$$\frac{\partial L}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right)\hat{y}(1-\hat{y}) = -y(1-\hat{y}) + (1-y)\hat{y} = \hat{y} - y.$$

The terms in $y\hat{y}$ cancel, leaving the simple form $\hat{y} - y$: the prediction error.

Step 4. Because $z = \sum_j w_j x_j + b$, we have $\partial z/\partial w_j = x_j$ and $\partial z/\partial b = 1$. Therefore

$$\frac{\partial L}{\partial w_j} = (\hat{y} - y)\, x_j, \qquad \frac{\partial L}{\partial b} = \hat{y} - y.$$

Step 5 (batch). J is the average of the per-example losses, and differentiation is linear, so

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m} (\hat{y}_i - y_i)\, x_{ij}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m} (\hat{y}_i - y_i).$$

In vector form the **gradient** is $\nabla_{\mathbf{w}} J = \frac{1}{m} X^{\mathsf{T}}(\hat{\mathbf{y}} - \mathbf{y})$, where row i of X is $\mathbf{x}_i^{\mathsf{T}}$. Each $\partial J/\partial w_j$ is a *partial derivative* (the other parameters are held fixed); together they form the gradient vector, which points toward the steepest *increase* of J.

A.6 Gradient descent parameter updates

To **minimise** J, move against the gradient:

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}, \qquad b \leftarrow b - \alpha \frac{\partial J}{\partial b}.$$

The learning rate $\alpha > 0$ sets the step size. For a small step, $J(\theta - \alpha g) \approx J(\theta) - \alpha \|g\|^2$ (first-order Taylor expansion, $g = \nabla J$), so J falls. If α is too small, training is slow; if too large, a step can overshoot and the loss can rise, so descent is not guaranteed to lower J at every single step. This is gradient *descent*; gradient *ascent* (a plus sign) would maximise an objective and is not used here.

Optional L2 term. The code can add a penalty: it then minimises $J + \frac{\lambda}{2}\|\mathbf{w}\|^2$, so the weight gradient becomes $\partial J/\partial w_j + \lambda w_j$ (the bias is not penalised). The pre-trained default uses $\lambda = 0.001$. Setting $\lambda = 0$ in the training dashboard gives exactly the equations above. The plotted loss is always J alone.

A.7 Worked numerical example

This hand calculation is illustrative and separate from the full invoice model. It uses a **synthetic** dataset with one feature and $m = 2$ examples: $(x_1, y_1) = (1, 0)$ and $(x_2, y_2) = (3, 1)$. Start at $w = 0,\ b = 0$ with $\alpha = 1$. (The numbers below are computed by the same functions as the playground on the Training page.)

Step	Calculation	Result
1. Linear part	$z_i = wx_i + b$	z₁ = 0.0, z₂ = 0.0
2. Probabilities	$\hat{y}_i = \sigma(z_i)$	ŷ₁ = 0.500000, ŷ₂ = 0.500000
3. Losses	$L_1 = -\ln(1 - \hat{y}_1),\ L_2 = -\ln \hat{y}_2$	L₁ = 0.693147, L₂ = 0.693147; J = 0.693147 (= ln 2)
4. Errors	$\hat{y}_i - y_i$	0.5 and -0.5
5. ∂J/∂w	$\frac{1}{m}\sum(\hat{y}_i - y_i)x_i$	(0.5·1 + (-0.5)·3)/2 = (0.5 + -1.5)/2 = -0.5000
6. ∂J/∂b	$\frac{1}{m}\sum(\hat{y}_i - y_i)$	(0.5 + -0.5)/2 = 0.0000
7. Step sizes	$\alpha\,\partial J/\partial w,\ \alpha\,\partial J/\partial b$	1 × (-0.5000) = -0.5000; 1 × 0.0000 = 0.0000
8. Update	$w \leftarrow w - \alpha g_w,\ b \leftarrow b - \alpha g_b$	w = 0 − (-0.5000) = 0.5000; b = 0 − 0.0000 = 0.0000
9. New predictions	$z_i = wx_i + b$	z₁ = 0.5, z₂ = 1.5; ŷ₁ = 0.622459, ŷ₂ = 0.817574
10. New loss	J	L₁ = 0.974077, L₂ = 0.201413; J = 0.587745 (down from 0.693147)

One update moved w in the direction that increases $\hat{y}_2$ (the true match) and decreased the loss from 0.693147 to 0.587745.

A.8 Training algorithm

```
input: X (m x n), y in {0,1}^m, learning rate alpha, iterations T, L2 strength lambda
standardise X using training mean and std            # mu, sigma stored
w <- 0 (n zeros);  b <- 0                             # initial loss = ln 2
for t = 0 .. T-1:
    y_hat <- sigmoid(X w + b)                         # prediction
    J     <- binary_cross_entropy(y, y_hat)           # recorded: loss[t]
    g_w   <- X^T (y_hat - y) / m + lambda * w         # gradients
    g_b   <- mean(y_hat - y)
    (w, b) <- (w - alpha*g_w, b - alpha*g_b)          # descent step
    if w or b not finite: stop, status = diverged
record J at the final (w, b)                          # loss[T]  -> T+1 values in all
return w, b, loss history
```

Stopping criteria: a fixed number of iterations (1 to 5000), or earlier if the parameters become non-finite (reported as *diverged*); an optional gradient-norm tolerance exists in the code but is off by default. **Reproducibility:** zero initialisation and a fixed seed for the synthetic data make every run identical.

A.9 Connection to the source code

Step	File	Function / class
Feature extraction	`backend/app/ml/features.py`	`vectorize()`
Pair similarities used as inputs	`backend/app/matching/engine.py`	`compare()`
Synthetic pairs, split by invoice	`backend/app/ml/synthetic.py`	`make_split(), featurize()`
Sigmoid	`backend/app/ml/model.py`	`sigmoid()`
Loss	`backend/app/ml/model.py`	`bce()`
Gradients	`backend/app/ml/model.py`	`gradients()`
Parameter update	`backend/app/ml/model.py`	`gd_step()`
Training loop	`backend/app/ml/model.py`	`GDLogisticRegression.fit()`
Training service and API	`backend/app/ml/service.py`, `backend/app/api/routes.py`	`train(), model_train()` (`POST /api/model/train`)
Default model (command line)	`backend/app/ml/train.py`	`main()`
Evaluation	`backend/app/ml/model.py`	`classification_metrics()`
Invoice-matching integration	`backend/app/services/runner.py`	`run()`
Dashboard and playground	`frontend/src/Training.tsx`, `frontend/src/math.ts`	`Training, Playground, gdTrace(), workedExample()`
Tests	`backend/tests/test_gd.py`	gradient check against finite differences, worked example, edge cases

In reconciliation, `run()` extracts the same features for each candidate pair, computes $\hat{y}$ with the active model, and records it as `model_probability`. It is shown separately from the *rule-based status*, the *fuzzy similarity score* (a weighted similarity, not a probability) and the *final status*, which the rules set and the user can accept or reject. If the rules say "Matched" but $\hat{y}$ is below the threshold, the row is flagged for manual review.

A.10 Model evaluation

On the held-out test pairs, with predicted match when $\hat{y} \geq$ threshold, let TP, FP, FN, TN be the confusion counts. The implementation (`classification_metrics()`) computes

$$\text{accuracy} = \frac{TP+TN}{m}, \quad \text{precision} = \frac{TP}{TP+FP}, \quad \text{recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision}+\text{recall}}$$

(a ratio with a zero denominator is reported as 0).

Measured values of the model currently active in Taxon (pre-trained default); these are read live from the backend, not typed in:

Quantity	Value
Data	SYNTHETIC invoice pairs
Training / test pairs	2098 / 899
Positive / negative (train)	700 / 1398
Positive / negative (test)	300 / 599
α, iterations, λ, threshold	0.5, 1500, 0.001, 0.5
Initial loss J	0.693147
Final loss J	0.066114
Accuracy	0.9566
Precision	1.0000
Recall	0.8700
F1	0.9305
Confusion (TN, FP, FN, TP)	599, 0, 39, 261
Bias b	-3.8558

Feature	Weight (standardised)
supplier	-0.2270
invoice	0.2660
date	1.8870
amount	1.2780
gstin	-0.0760
supplier_missing	0.0000
invoice_missing	0.0000
date_missing	0.6290
amount_missing	0.0000
gstin_missing	-0.0730
amount_rel_diff	-1.2780
date_diff_scaled	-2.3360
invoice_exact	2.9890

These figures come from synthetic data that the author generated. They show that the training pipeline works; they do **not** show how the model would perform on real GST invoice records.

A.11 Limitations and conclusion

Synthetic data: the labels and the noise are designed, so real records may differ in ways not represented. **Noisy records and class imbalance:** real data would contain more near-duplicates and far more non-matches than matches; accuracy alone could then mislead, which is why precision, recall and F1 are reported. **Features:** the model sees only the listed similarities, so information outside them is invisible to it. **Learning rate and threshold:** too large an α can make the loss oscillate or rise; the threshold trades precision against recall. **Manual review:** in financial reconciliation a wrong automatic match has a cost, so the model supports, and never replaces, a person's decision.

Conclusion. The loss J measures how far predicted probabilities are from the true labels. Its derivatives, found with the chain rule, give the gradient $\frac{1}{m}X^{\mathsf{T}}(\hat{\mathbf{y}} - \mathbf{y})$, which tells Taxon how a small change in each weight changes the loss. Repeatedly stepping against that gradient is gradient descent, and it is how Taxon learns its model parameters by minimising prediction loss.